

DECORATOR APLICADO EN TARIFAS DE TAXI

Patrones de Diseño: *Decorator*

Adición dinámica de responsabilidades y
recargos a un objeto de tarifa base

Autor: **Angie Nathaly Cisneros Guerrero**

20242020084

Fecha: Septiembre 2026

Índice

1. Diagrama UML	2
2. Explicación de las Clases	3
2.1. Componente y Componente Concreto	3
2.2. Decorador Base (DecoratorTarifa)	3
2.3. Decoradores Concretos (Recargos)	5
3. Código de Cliente	7
3.1. Clase Main	7
4. Pruebas y Ejecución	8
4.1. Prueba de Ejecución en Consola	8
5. Repositorio	9
5.1. Enlace GitHub	9

Diagrama UML

Descripción General

El proyecto implementa el patrón de diseño **Decorator**, un patrón estructural que permite añadir funcionalidades y comportamientos (en este caso, recargos de tarifa) a un objeto dinámicamente. Esto se logra envolviendo el objeto original dentro de clases decoradoras que comparten la misma interfaz, ofreciendo una alternativa flexible a la herencia para extender la funcionalidad.

Diagrama de Clases

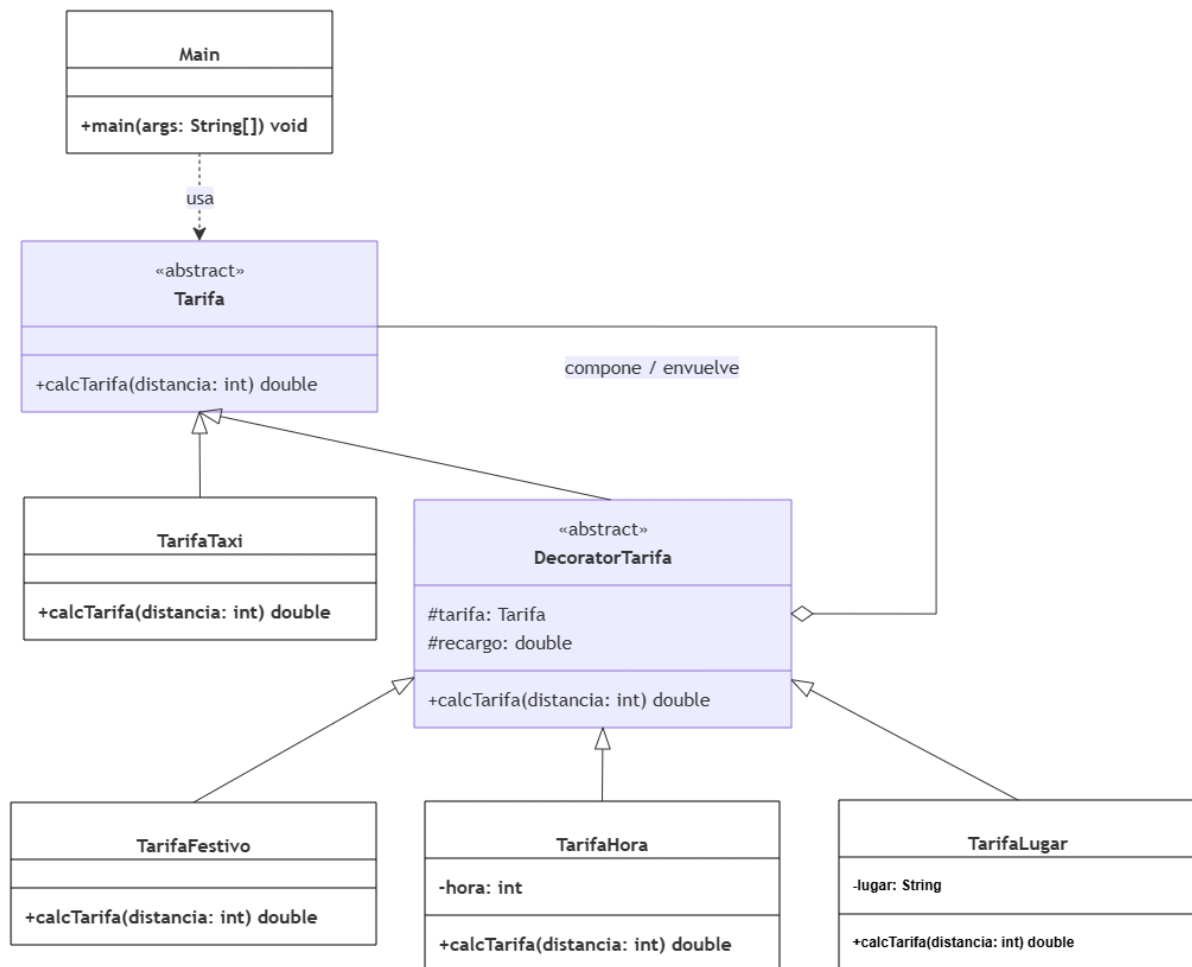


Figura 1: Diagrama de Clases del Sistema (Patrón Decorator).

Explicación de las Clases

Componente y Componente Concreto

Descripción

Tarifa es la clase abstracta (Componente) que define la interfaz común para la tarifa base y sus decoradores. **TarifaTaxi** es la implementación concreta (Componente Concreto) que define el comportamiento por defecto, es decir, el cobro base del servicio por distancia.

Código Fuente: Tarifa.java

```
1 package com.mycompany.tarifastaxi.decorator;
2
3 public abstract class Tarifa {
4
5     public abstract double calcTarifa(int distancia);
6
7 }
```

Código Fuente: TarifaTaxi.java

```
1 package com.mycompany.tarifastaxi.decorator;
2
3 public class TarifaTaxi extends Tarifa{
4
5     @Override
6     public double calcTarifa(int distancia) {
7         return distancia * 1000;
8     }
9
10 }
```

Decorator Base (DecoratorTarifa)

Descripción

La clase abstracta **DecoratorTarifa** mantiene una referencia a un objeto **Tarifa** y delega las operaciones a este objeto. Es la clase base de la cual heredan todos los decoradores específicos.

Código Fuente: DecoratorTarifa.java

```
1 package com.mycompany.tarifastaxi.decorator;
2
3 public abstract class DecoratorTarifa extends Tarifa{
4
5     protected Tarifa tarifa;
6     protected double recargo;
7
8     public DecoratorTarifa(Tarifa tarifa) {
9         this.tarifa = tarifa;
10 }
```

```
11
12     @Override
13     public double calcTarifa(int distancia) {
14         return tarifa.calcTarifa(distancia);
15     }
16
17 }
```

Decoradores Concretos (Recargos)

Descripción

Son las clases que heredan del `DecoratorTarifa`. Sobrescriben el método `calcTarifa` para llamar al cálculo del objeto envuelto y sumarle su respectivo recargo lógico (por festivo, hora o lugar).

Código Fuente: `TarifaFestivo.java`

```
1 package com.mycompany.tarifastaxi.decorator;  
2  
3 public class TarifaFestivo extends DecoratorTarifa{  
4  
5     public TarifaFestivo(Tarifa tarifa) {  
6         super(tarifa);  
7     }  
8  
9     @Override  
10    public double calcTarifa(int distancia) {  
11        double recargo = 5000;  
12        return super.calcTarifa(distancia) + recargo;  
13    }  
14  
15 }
```

Código Fuente: `TarifaHora.java`

```
1 package com.mycompany.tarifastaxi.decorator;  
2  
3 public class TarifaHora extends DecoratorTarifa{  
4     private int hora;  
5  
6     public TarifaHora( Tarifa tarifa, int hora) {  
7         super(tarifa);  
8         this.hora = hora;  
9     }  
10  
11    @Override  
12    public double calcTarifa(int distancia) {  
13        if ((hora >= 22 && hora <= 24) || (hora >= 1 && 3 <= hora  
14            )){  
15            this.recargo = 3000;  
16        }  
17        return super.calcTarifa(distancia) + recargo;  
18    }  
19 }
```

Código Fuente: `TarifaLugar.java`

```
1 package com.mycompany.tarifastaxi.decorator;  
2  
3 public class TarifaLugar extends DecoratorTarifa{  
4     private String lugar;  
5     public TarifaLugar(Tarifa tarifa, String lugar) {  
6         super(tarifa);
```

```
7         this.lugar = lugar;
8     }
9
10    @Override
11    public double calcTarifa(int distancia) {
12        double recargoLugar = 0;
13        if(lugar.equalsIgnoreCase("Aeropuerto")){
14            recargoLugar = 5000;
15        }else if (lugar.equalsIgnoreCase("estadio")){
16            recargoLugar = 3000;
17        }else if(lugar.equalsIgnoreCase("centro_comercial")){
18            recargoLugar = 1000;
19        }
20
21        return super.calcTarifa(distancia) + recargoLugar;
22    }
23
24 }
```

Código de Cliente

Clase Main

Descripción

La clase **Main** actúa como el **Cliente**. Es responsable de instanciar la tarifa base (**TarifaTaxi**) y de envolverla dinámicamente con los diferentes decoradores de recargo según las condiciones dadas, acumulando el costo final.

Código Fuente: Main.java

```
1 package com.mycompany.tarifastaxi.decorator;
2
3 public class Main {
4
5     public static void main(String[] args) {
6
7         Tarifa tarifa = new TarifaTaxi();
8
9         tarifa = new TarifaHora(tarifa, 2);
10        tarifa = new TarifaLugar(tarifa, "Aeropuerto");
11
12        int distanciaKm = 10;
13        System.out.println("Total: ␣$" + tarifa.calcTarifa(
14            distanciaKm));
15    }
```

Flujo de Ejecución

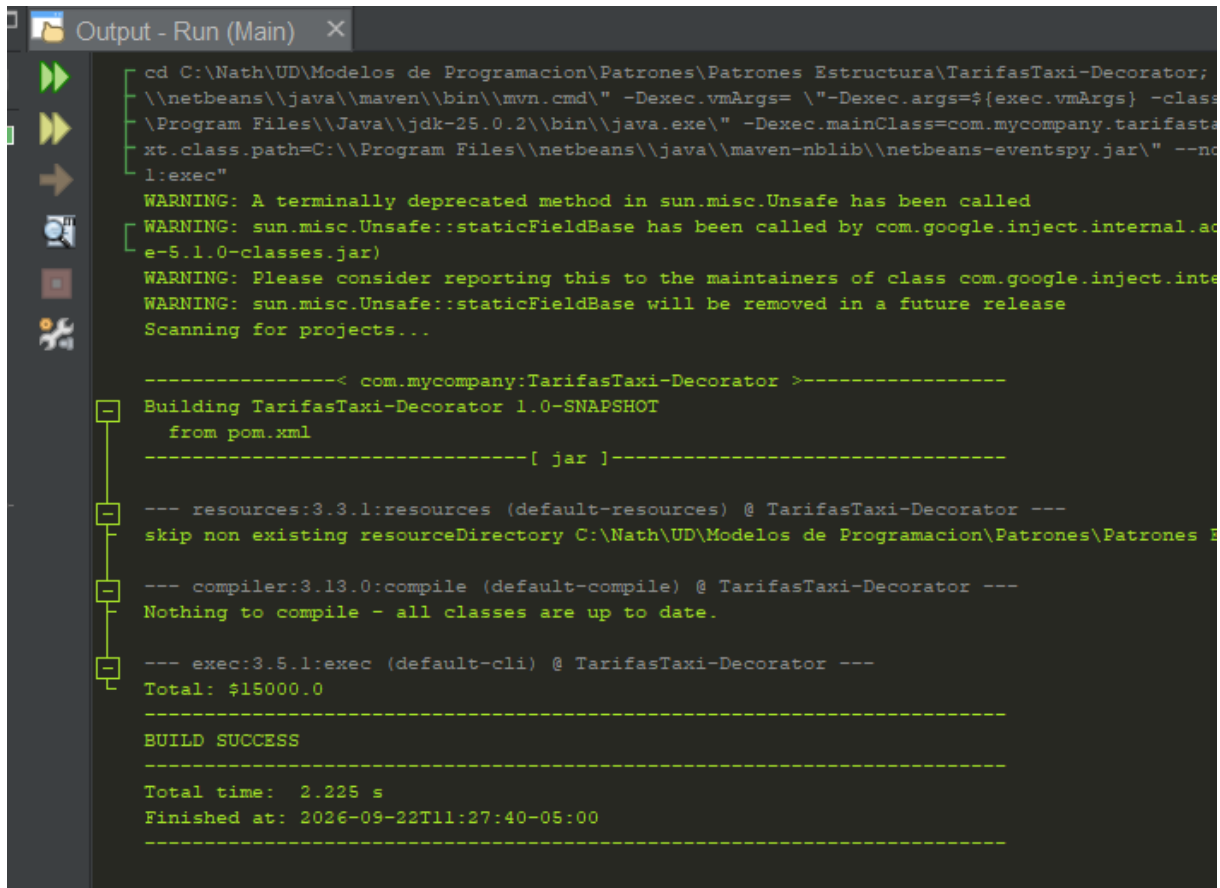
1. Se instancia la **TarifaTaxi** que representa el cobro base por distancia.
2. Se envuelve la tarifa original dentro de un decorador **TarifaHora** pasándole la hora 2 (madrugada), lo que agregará un recargo.
3. Se envuelve nuevamente el objeto resultante dentro del decorador **TarifaLugar** para el destino "Aeropuerto", apilando otro recargo.
4. Se invoca **calcTarifa(10)**. La llamada viaja por la cadena de decoradores acumulando los montos.

Pruebas y Ejecución

Prueba de Ejecución en Consola

Al ejecutar la clase Main con una distancia de 10 Km a la hora 2 hacia el "Aeropuerto", la salida obtenida por consola refleja la suma acumulada correctamente delegada a través del patrón Decorator:

Total: \$18000.0



```
cd C:\Nath\UD\Modelos de Programacion\Patrones\Patrones Estructura\TarifasTaxi-Decorator;
\\netbeans\java\maven\bin\mvn.cmd" -Dexec.vmArgs= "-Dexec.args=${exec.vmArgs} -class
\Program Files\Java\jdk-25.0.2\bin\java.exe" -Dexec.mainClass=com.mycompany.tarifast
xt.class.path=C:\Program Files\netbeans\java\maven-nblib\netbeans-eventspy.jar" --no
l:exec"
WARNING: A terminally deprecated method in sun.misc.Unsafe has been called
WARNING: sun.misc.Unsafe::staticFieldBase has been called by com.google.inject.internal.ac
e-5.1.0-classes.jar)
WARNING: Please consider reporting this to the maintainers of class com.google.inject.inte
WARNING: sun.misc.Unsafe::staticFieldBase will be removed in a future release
Scanning for projects...

-----< com.mycompany:TarifasTaxi-Decorator >-----
Building TarifasTaxi-Decorator 1.0-SNAPSHOT
from pom.xml
-----[ jar ]-----

--- resources:3.3.1:resources (default-resources) @ TarifasTaxi-Decorator ---
skip non existing resourceDirectory C:\Nath\UD\Modelos de Programacion\Patrones\Patrones E

--- compiler:3.13.0:compile (default-compile) @ TarifasTaxi-Decorator ---
Nothing to compile - all classes are up to date.

--- exec:3.5.1:exec (default-cli) @ TarifasTaxi-Decorator ---
Total: $15000.0
-----
BUILD SUCCESS
-----
Total time: 2.225 s
Finished at: 2026-09-22T11:27:40-05:00
-----
```

Figura 2: Salida Consola).

Repositorio

Enlace GitHub

- <https://github.com/Thalyhx/Modelos-de-Programacion/tree/main/Patrones%20Estructura/TarifasTaxi-Decorator>